

★ Author

Name: Bipin Tiwari

Roll number: DS24f3001090

Student Email: 24f3001090@ds.study.iitm.ac.in

I am in the third term of diploma level and I took MAD 2 theory + project in this term. I am pursuing one more degree other than this from Jabalpur Engineering College and that is a B.Tech program in AI & DS.

★ AI/LLM usage

I used **Gemini** to understand the frameworks and concepts needed to be applied in the given application. I also used Gemini to help me design the UI in the application.

The overall usage of AI/LLM in my web app would be around 25%.

★ Description

In the placement portal application, 3 kinds of dashboards were needed to be created and role based access was needed to be provided to the users. Admin or college must approve companies' registrations to give them access and admin can blacklist any student or company or manage any drive or job application. Companies can create placement drives and students can apply for jobs. Besides, scheduled and triggered jobs are also implemented with the help of celery and redis. Redis is also used for caching.

★ Technologies used

- **Flask:** The main backend web framework
- **SQLite:** Database for storing data
- **SQLAlchemy:** Object Relational Mapper for SQLite Database
- **Vue JS:** Frontend framework
- **Bootstrap:** Used for Frontend styling
- **Celery:** Scheduled and triggered backend jobs
- **Redis:** Caching and message broker for celery

★ DB Schema Design

Following are the tables in the database:

- **Student:** stores student details (s_id, name, email, password, level, cgpa, blacklisted, resume)
- **Company:** stores company details (c_id, name, email, password, blacklisted, status, blacklisted)
- **Drive:** contains details about job positions (d_id, role, package, experience, status, company_id)
- **Application:** contains details about job applications (a_id, status, app_date, int_date, student_id, drive_id)
- **Skill:** contains details about skills required for job positions (s_id, name, drive_id)

Following are the relations in database:

- **Student -> Application:** One-to-Many
- **Company -> Drive:** One-to-Many
- **Drive -> Skill:** Many-to-Many
- **Application -> Drive:** One-to-Many

The tables and relations are created in this way because it helps to implement required features and at the same time the database is simple to create and understand.

★ Architecture and Features

Architecture:

Backend:

- **app.py:** main application python file
- **dbmodel.py:** contains database tables and schema
- **celery_worker.py:** contains a celery worker for backend jobs
- **tasks.py :** contains backend tasks
- **static/exports:** contains exported csv
- **/uploads/resumes:** folder to store resumes
- **/templates:** HTML template of report
- **/instance:** contains sqlite3 database

Frontend:

- **public/:** contains image and favicon
- **src/:** contains vue components, views and assets used in frontend styling.
- **Other configuration files.**

Features:

- Student and Company registration and login.
- Creation of placement drives and selection/rejection job applications by companies.
- Application for roles and its tracking by students.
- Searching for job positions by students.
- Profile management for the student's dashboard.
- Admin approval for company registrations.
- Management/blacklisting/deactivating users/drives/applications by Admin.
- Deletion of drive by admin.
- Searching Students or Companies by admin.
- Session handling and logout.
- Scheduled backend jobs of interview reminders or monthly placement report through emails.
- Caching for API optimization.

★ Video**Link:**

<https://drive.google.com/file/d/1EqxqncXDh2-2swc7bHqMzWajSxBam24u/view?usp=sharing>